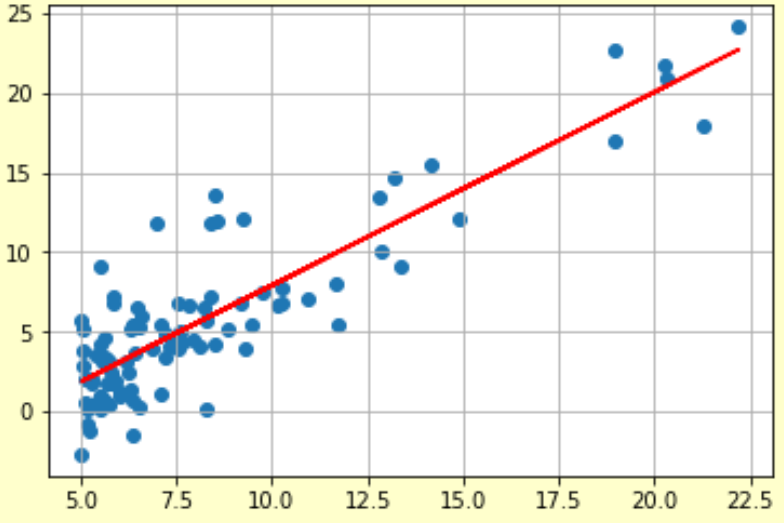
Régression linéaire

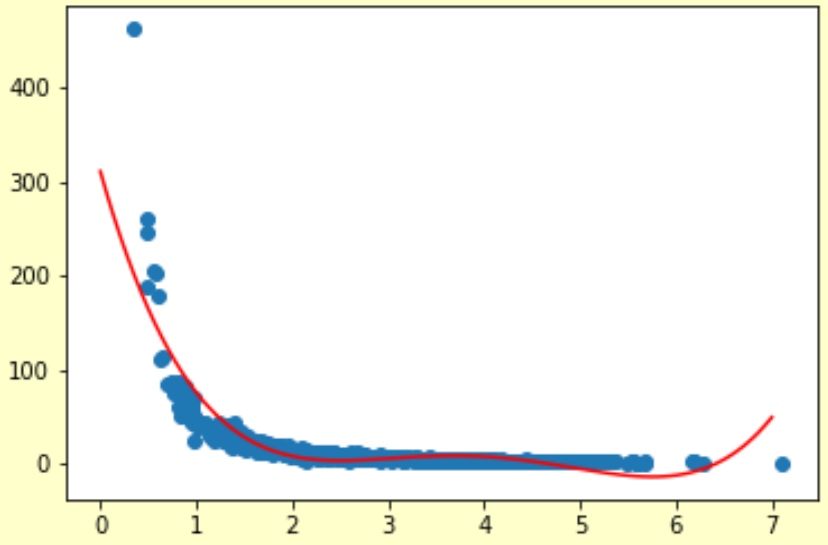
Les algorithmes de régression linéaire modélisent la relation entre des variables prédictives et une variable cible. La relation est modélisée par une fonction de prédiction. Le cas le plus simple est la régression linéaire univariée. Elle va trouver une fonction sous forme de droite pour estimer la relation.

La régression polynomiale permet de modéliser des relations complexes qui ne sont pas forcément linéaires.

Relation linéaire entre des variables prédictives et la variable cible



Relation non linéaire entre des variables prédictives et la variable cible



38

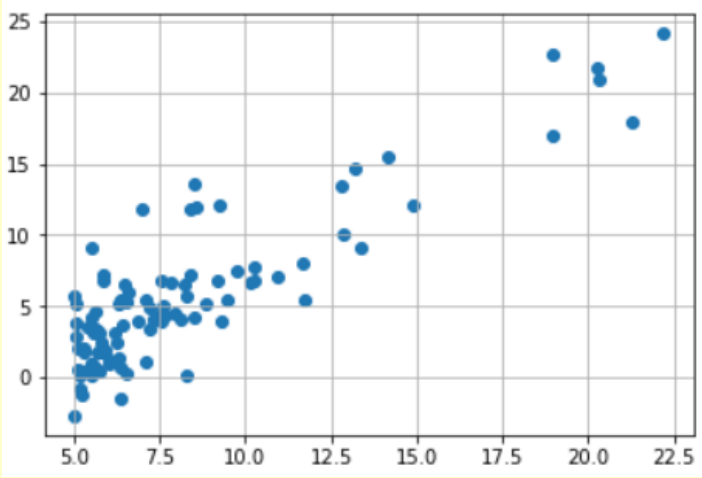
Régression linéaire

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
df = pd.read_csv("d:\\univariate_linear_regression_dataset.csv")
X =  df.iloc[:,0]
Y =  df.iloc[:,1]
axes = plt.axes()
axes.grid()
plt.scatter(X,Y)
plt.show()
SL=stats.linregress(X, Y)
slope=SL.slope; intercept=SL.intercept; coef_correlation=SL.rvalue
def predict(x):
    return slope * x + intercept
axes = plt.axes()
axes.grid()
plt.scatter(X,Y)
fitLine = predict(X)
plt.plot(X, fitLine, c='r')
plt.show()
```

39

Régression linéaire

```
axes = plt.axes()
axes.grid()
plt.scatter(X,Y)
plt.show()
```



Régression linéaire

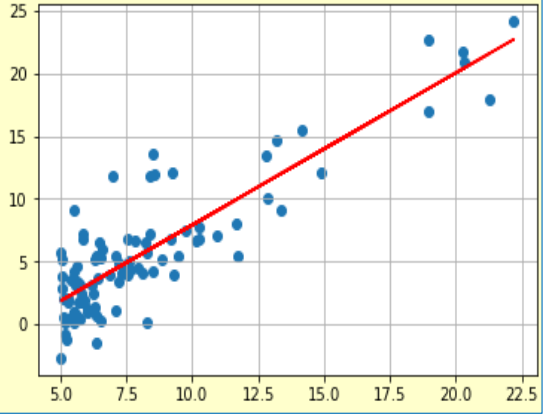
- Le fonction de prédiction pour une régression linéaire univariée est comme suit :

$$H(x)=intercept+slope*x$$

- avec : $slope$: représente la "pente" de la ligne de prédiction et $intercept$ représente le point d'intersection avec l'axe des ordonnées

SL= stats.linregress(X, Y)

```
SL.slope est 1.2135472539083585
SL.intercept est -4.211504005424089
SL.rvalue:
0.8721572919685905
r= X.corr(Y)
print(r)
0.8721572919685905
```



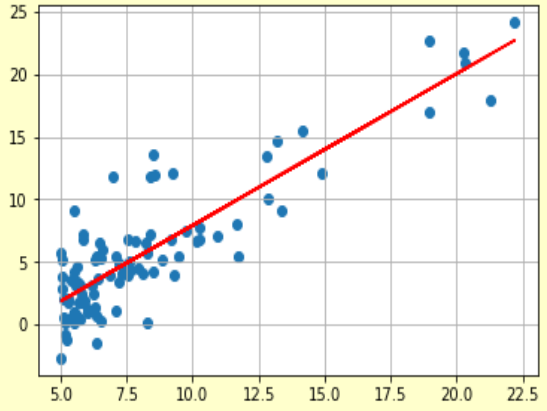
Régression linéaire

- **Prédiction d'une nouvelle observation:**

  On voit que pour la valeur x = 22.5, la valeur de y est environ 25.

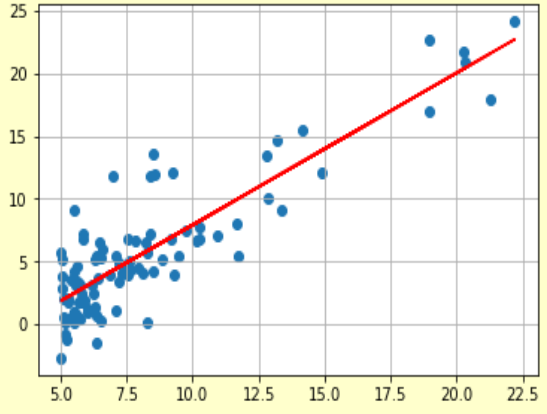
  Utilisons la fonction $predict$ pour trouver une estimation de $H(x$=22.5):

  predict(22.5) ➔ 23.093309207513975



Régression linéaire avec l'algorithme de descente de gradient

Dans la pratique, les Data Scientists utilisent le package **sklearn**, qui permet d'écrire un tel code en 4 lignes, mais ici nous écrirons chaque fonction mathématique de façon explicite, ce qui est un **bon exercice** pour la compréhension du modèle.



Régression linéaire avec l'algorithme de descente de gradient

$$X = \begin{pmatrix} x & 1 \end{pmatrix} \quad ; \quad \theta = \begin{pmatrix} a \\ b \end{pmatrix} \quad et \quad f(x) = \theta.X = \begin{pmatrix} x & 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = a.x + b$$

Objectif: trouver θ qui donne une bonne prédiction d'une entrée x.

Pour développer un modèle linéaire avec la descente de gradient, il faut implémenter les 4 fonctions clefs suivantes :

- la fonction de notre modèle : $f(x) = X.\theta$
- la fonction Cout : $J(\theta) = \frac{1}{2m} \sum (X.\theta - Y)^2$
- le gradient : $\frac{\partial J(\theta)}{\partial \theta} = \frac{1}{m} X^T.(X.\theta - Y)$
- la descente de gradient : $\theta = \theta - \alpha \times \frac{\partial J(\theta)}{\partial \theta}$

α: est appelé taux d'apprentissage: learning_rate

m: est la taille de l'ensemble d'apprentissage

Objectif: trouver θ qui minimise J(θ)

44

Régression linéaire avec l'algorithme de descente de gradient

- la fonction de notre modèle : $f(x) = X.\theta$
- la fonction Cout : $J(\theta) = \frac{1}{2m} \sum (X.\theta - Y)^2$
- le gradient : $\frac{\partial J(\theta)}{\partial \theta} = \frac{1}{m} X^T.(X.\theta - Y)$
- la descente de gradient : $\theta = \theta - \alpha \times \frac{\partial J(\theta)}{\partial \theta}$

```
def f(X, theta):
    return X.dot(theta)
def erreur_somme_des_distances(X, y, theta):
    m = len(y)
    return (1/(2*m))*np.sum((f(X, theta)-y)**2)
def gradient(X, y, theta):
    m = len(y)
    return 1/m * X.T.dot(f(X, theta) - y)
```

45

Régression linéaire avec l'algorithme de descente de gradient

- la fonction de notre modèle : $f(x) = X.\theta$
- la fonction Cout : $J(\theta) = \frac{1}{2m}\sum(X.\theta - Y)^2$
- le gradient : $\frac{\partial J(\theta)}{\partial \theta} = \frac{1}{m}X^T.(X.\theta - Y)$
- la descente de gradient : $\theta = \theta - \alpha \times \frac{\partial J(\theta)}{\partial \theta}$

```
def gradient_descent(X, y, theta, learning_rate, n_iterations):
    # création d'un tableau de stockage pour enregistrer l'évolution des er
    historique_des_erreurs = np.zeros(n_iterations)
    for i in range(0, n_iterations):
        theta = theta - learning_rate * gradient(X, y, theta) # mise a jour
        historique_des_erreurs[i] = erreur_somme_des_distances(X, y, theta)
    return theta, historique_des_erreurs
```


Régression linéaire avec l'algorithme de descente de gradient

```
import  numpy as np
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv("d:\\univariate_linear_regression_dataset.csv")
x=  df.iloc[:,0] ; n_samples = len(x); x=np.array(x).reshape((n_samples, 1)) #96
y=  df.iloc[:,1] ; y=np.array(y).reshape((n_samples, 1))
x=np.array(x).reshape((n_samples, 1))
plt.scatter(x, y) # afficher les résultats. X en abscisse et y en ordonnée
plt.show()
# ajout de la colonne de biais a X
X = np.hstack((x, np.ones(x.shape)))
# création d'un vecteur parametre theta
theta = np.random.randn(2, 1)
print(theta)
```

Régression linéaire avec l'algorithme de descente de gradient

```
def f(X, theta):
   return X.dot(theta)
def erreur_somme_des_distances(X, y, theta):
   m = len(y)
   return (1/(2*m))*np.sum((f(X, theta)-y)**2)
def gradient(X, y, theta):
   m = len(y)
   return 1/m * X.T.dot(f(X, theta) - y)
def gradient_descent(X, y, theta, learning_rate, n_iterations):
   # création d'un tableau de stockage pour enregistrer l'évolution des erreurs
   historique_des_erreurs = np.zeros(n_iterations)
   for i in range(0, n_iterations):
      theta = theta - learning_rate * gradient(X, y, theta) # mise a jour du theta
      historique_des_erreurs[i] = erreur_somme_des_distances(X, y, theta)
   return theta, historique_des_erreurs
```

48

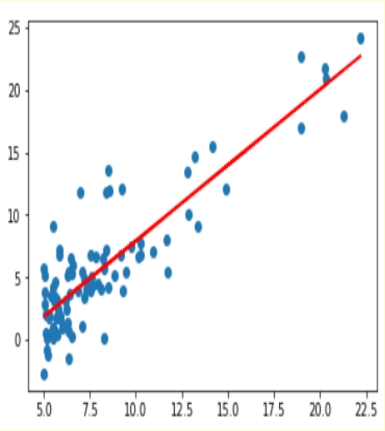
Régression linéaire avec l'algorithme de descente de gradient

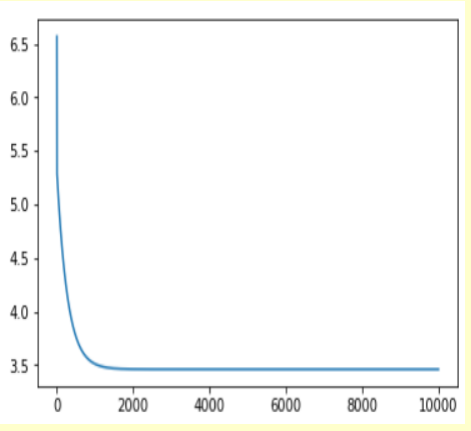
```
# Example de test :
n_iterations = 10000
learning_rate = 0.01
theta_final, historique_des_erreurs=gradient_descent(X, y, theta,
    learning_rate, n_iterations)
print(theta_final) # theta une fois que la machine a  été entrainée
 # création d'un vecteur prédictions qui contient les prédictions de notre modele final
predictions = f(X, theta_final)
 # Affiche les résultats de prédictions (en rouge) par rapport a notre Dataset (en bleu)
plt.scatter(x, y)
plt.plot(x, predictions, c='r')
plt.show()
plt.figure()
plt.plot(range(n_iterations), historique_des_erreurs)
plt.show()
print(f(np.array([22.5,1]), theta_final))
```

49

Régression linéaire avec l'algorithme de descente de gradient

f(np.array([22.5,1]), theta_final) ➔ donne: array([23.09330913])





Régression linéaire avec l'algorithme de descente de gradient

$f(x) = x^2 - x + 1 \Rightarrow \nabla f(x) = \frac{\partial f(x)}{\partial x} = f'(x) = 2x - 1$

Minimum au point x: f(x)=0
X=0.5

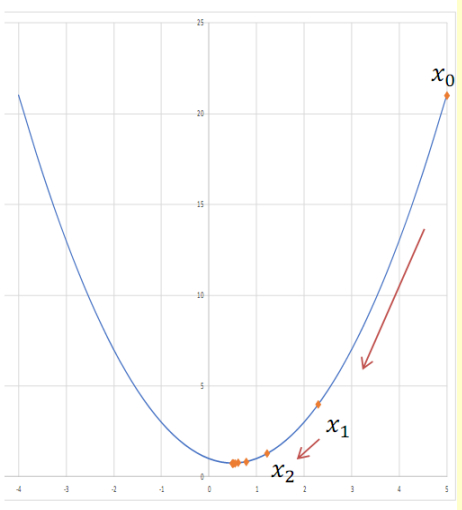
$\eta = 0.3$

eta	0.3

$x_0 = 5$

x	f'(x_t)	f(x)
5.0000		21.0000
2.3000	9.0000	3.9900
1.2200	3.6000	1.2684
0.7880	1.4400	0.8329
0.6152	0.5760	0.7633
0.5461	0.2304	0.7521
0.5184	0.0922	0.7503
0.5074	0.0369	0.7501
0.5029	0.0147	0.7500
0.5012	0.0059	0.7500
0.5005	0.0024	0.7500
0.5002	0.0009	0.7500
0.5001	0.0004	0.7500
0.5000	0.0002	0.7500

$x_{t+1} = x_t - \eta \times \nabla f(x_t)$



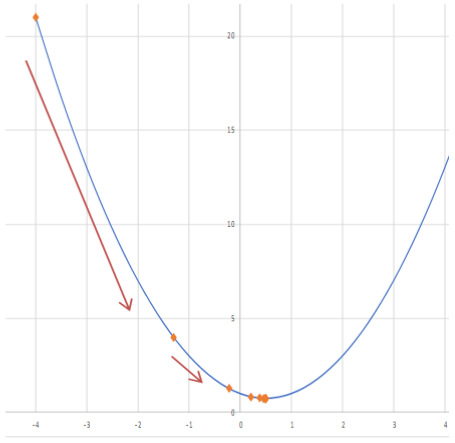
Régression linéaire avec l'algorithme de descente de gradient

$f(x) = x^2 - x + 1 \Rightarrow \nabla f(x) = \frac{\partial f(x)}{\partial x} = f'(x) = 2x - 1$

Minimum au point x: f(x)=0
X=0.5

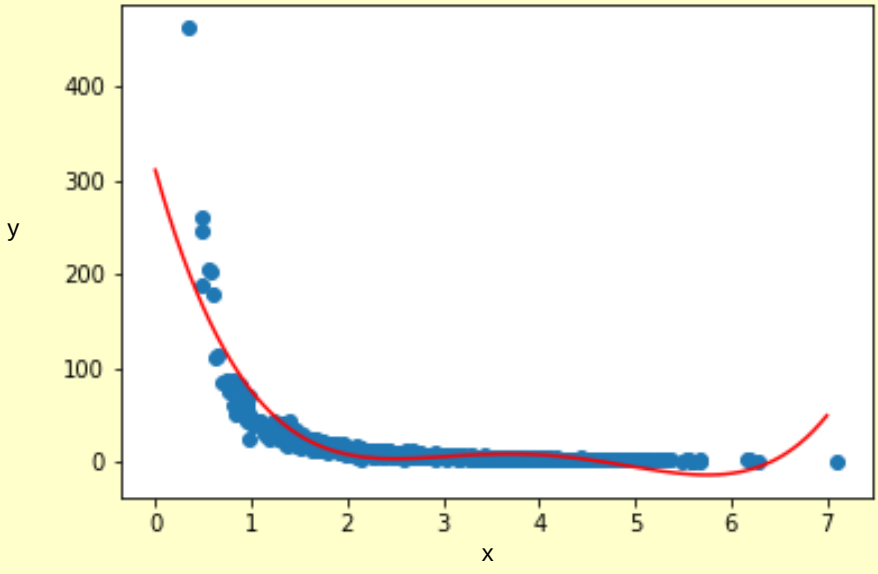
eta	0.3

x	f'(x_t)	f(x)
-4.0000		21.0000
-1.3000	-9.0000	3.9900
-0.2200	-3.6000	1.2684
0.2120	-1.4400	0.8329
0.3848	-0.5760	0.7633
0.4539	-0.2304	0.7521
0.4816	-0.0922	0.7503
0.4926	-0.0369	0.7501
0.4971	-0.0147	0.7500
0.4988	-0.0059	0.7500
0.4995	-0.0024	0.7500
0.4998	-0.0009	0.7500
0.4999	-0.0004	0.7500
0.5000	-0.0002	0.7500



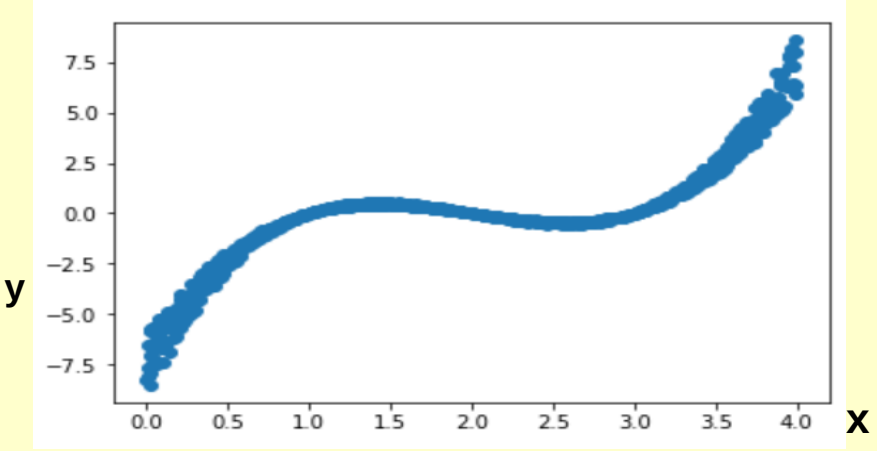
$x_{t+1} = x_t - \eta \times \nabla f(x_t)$

Régression polynomiale:
Relation non linéaire entre des variables prédictives et la variable cible



53

Régression polynomiale:
Relation non linéaire entre des variables prédictives et la variable cible



L'objectif ici c'est de trouver un polynôme P:P(X)≈y

Pour une régression polynomiale, la puissance de certaines variables indépendantes est supérieure à 1: Y = $a_n \cdot x^n + a_{n-1} \cdot x^{n-1} + \ldots + a_1 \cdot x^1 + a_0 \cdot x^0$

54

Régression polynomiale:
Relation non linéaire entre des variables prédictives et la variable cible
Régression polynomiale

```
def afficher_pol(LPol):
    L=LPol[:]; L.reverse()
    L=[round(e,2) for e in L]
    for i in range(len(L)):
        print('('+str(L[i])+'*X^'+str(i)+')',end='+')
def Evaluer_pol(LPol,x):
    s=0
    L=LPol[:]
    L.reverse()
    for i in range(len(LPol)):
        s+=L[i]*(x**i)
    return(s)
```

```
P=[-4.04264497e-03, 1.26341942e+00, -7.46569108e+00,
1.36046931e+01, -7.39512820e+00]
```

Afficher_pol(P)➔(-7.4*X^0)+(13.6*X^1)+(-7.47*X^2)+(1.26*X^3)+(-0.0*X^4)

55

Régression polynomiale:
Relation non linéaire entre des variables prédictives et la variable cible

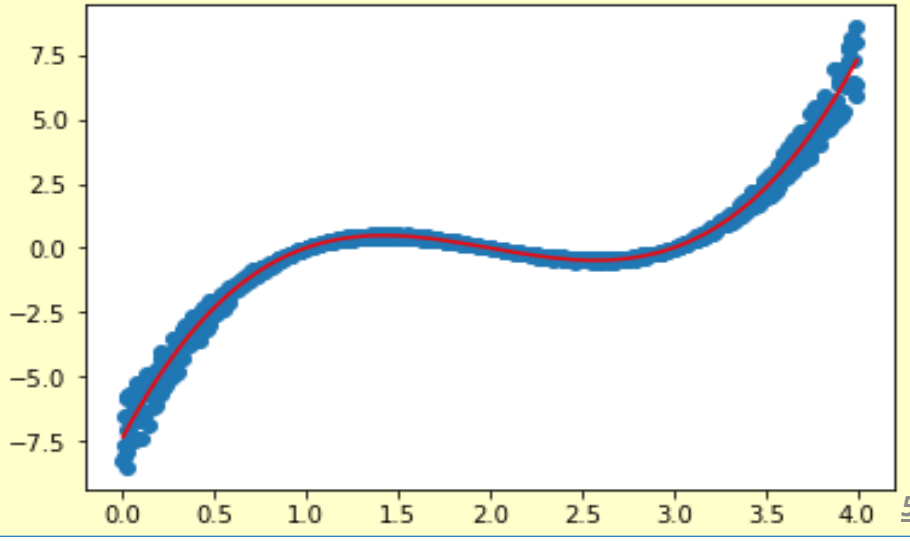
```
import numpy as np
import matplotlib.pyplot as plt
import pandas
data=pandas.read_csv("d:\\dataset_polynomial3.csv").as_matrix()
x=data[1:1001,0]
y=data[1:1001,1]
degre_polynome=5
p= np.poly1d(np.polyfit(x, y,degre_polynome))
LPol=list(p)
afficher_pol(LPol)
les_x= np.linspace(min(x), max(x), 100)
plt.scatter(x, y)
plt.plot(les_x, p(les_x), c='r')
plt.show()
```

P(x)=(-7.36*X^0)+(13.35*X^1)+(-7.01*X^2)+(0.96*X^3)+(0.08*X^4)+(-0.01*X^5)

56

Régression polynomiale:
Relation non linéaire entre des variables prédictives et la variable cible

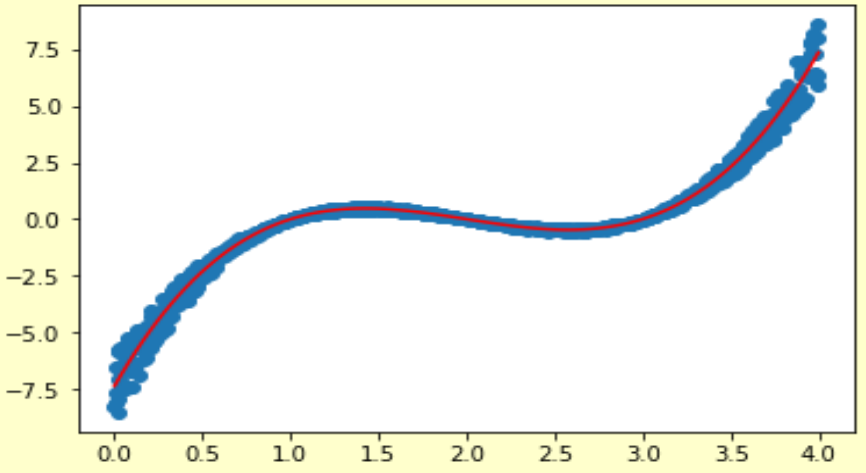
Polynôme de degré 5:
P(x)=(-7.36*X^0)+(13.35*X^1)+(-7.01*X^2)+(0.96*X^3)+(0.08*X^4)+(-0.01*X^5)



57

Régression polynomiale:
Relation non linéaire entre des variables prédictives et la variable cible

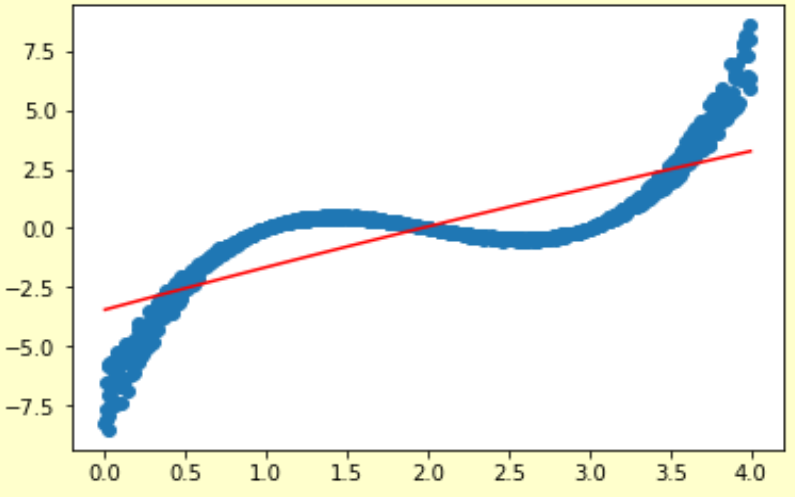
Polynôme de degré 3:
P(x)=(-7.38*X^0)+(13.53*X^1)+(-7.38*X^2)+(1.23*X^3)



58

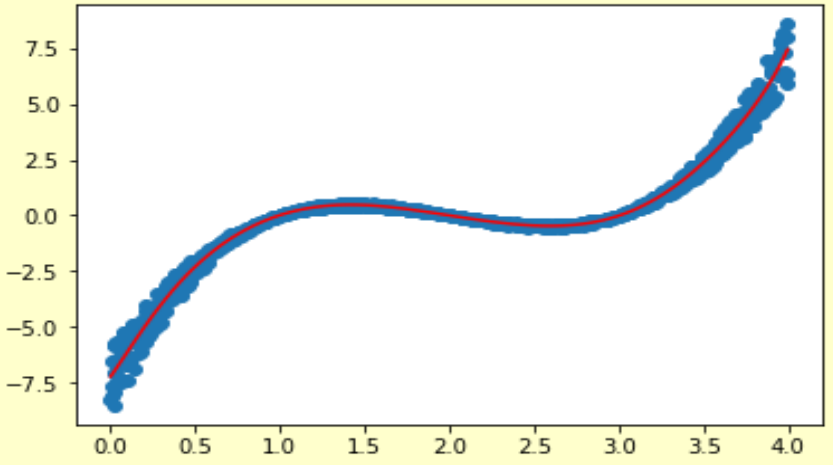
Régression polynomiale:
Relation non linéaire entre des variables prédictives et la variable cible

Polynôme de degré 2:
P(x)=(-3.45*X^0)+(1.84*X^1)+(-0.04*X^2)



59

Régression polynomiale:
Relation non linéaire entre des variables prédictives et la variable cible

Polynôme de degré 10:

P(x)=(-7.24*X^0)+(10.79*X^1)+(6.73*X^2)+(-32.93*X^3)+(47.14*X^4)+(-40.23*X^5)+(21.92*X^6)+(-7.62*X^7)+(1.63*X^8)+(-0.2*X^9)+(0.01*X^10